

DevOps Assignment: Full CI/CD Pipeline

Node.js backend · ESLint · Unit tests · Docker · GitLab CI/CD · AWS EC2

Type: Individual project | **Estimated effort:** 12–20 hours | **Submission:** GitLab project URL + short report

1. Goal

You will build a small backend application and wrap it in a complete, working delivery pipeline. The application itself is deliberately simple — **the pipeline is the actual assignment**. By the end you must be able to open a merge request, watch it deploy automatically to a development server, merge it, and watch it deploy automatically to a production server. Both servers are real AWS EC2 instances, and the job that deploys to them runs on a GitLab Runner that you installed and registered yourself.

What you will practise

- Writing testable backend code with linting and unit tests wired into automation.
- Containerising an application and publishing images to a private container registry.
- Provisioning Linux servers on AWS and configuring secure SSH access for a machine user.
- Installing and registering a self-hosted GitLab Runner.
- Designing a multi-stage pipeline with branch-dependent behaviour.
- Managing secrets correctly: environment-scoped, protected, and masked CI/CD variables.

2. Deliverables

Deliverable	Requirement
GitLab repository	Public, or private with the instructor added as Maintainer . Must contain the app, tests, ESLint config, <code>Dockerfile</code> and <code>.gitlab-ci.yml</code> .
Green pipeline on <code>main</code>	All stages passing. The pipeline history must not be cleared — failed attempts are fine and expected.
At least one merged MR	Must show a dev deployment job that ran on the MR, and a prod deployment that ran after merge.
Two live URLs	Dev and prod endpoints reachable over HTTP, returning different environment identifiers.
Images in registry	At least three tagged images visible in your project's GitLab Container Registry.
<code>README.md</code>	Setup instructions, architecture description, the two URLs, and a list of every CI/CD variable you created (names and purpose only — never values).
Report (1–2 pages, PDF)	What you built, what broke, how you fixed it. Include screenshots of your pipeline graph, your runner in the Runners page, and your CI/CD variables list.

Cost warning. AWS charges for running instances. Use `t3.micro` or `t2.micro` and stay inside the Free Tier where possible. **Stop or terminate all three instances as soon as the assignment is graded.** Set a billing alarm on day one. You are responsible for your own AWS bill.

3. Part A — The Backend Application

A.1 Requirements

- Node.js (LTS), JavaScript only. Any HTTP framework is fine (Express, Fastify, Koa, or plain `http`).
- Listens on a port taken from the `PORT` environment variable, defaulting to `3000`.
- Reads an `APP_ENV` environment variable so the running instance can identify itself.

Required endpoints

Method	Path	Behaviour
GET	<code>/health</code>	Returns <code>200</code> and <code>{"status":"ok"}</code> . Used by you and by your deploy script to verify a release.
GET	<code>/version</code>	Returns the app version, the environment name, and the Git commit SHA that was baked into the image at build time.

Plus a small resource of your choice — for example an in-memory list of items with `GET`, `POST`, `GET /:id` and `DELETE /:id`. It must contain at least one piece of non-trivial logic (validation, filtering, or a calculation) that is worth unit testing.

Why `/version` matters. After a deployment you need to prove that the *new* code is actually running. The commit SHA is the cleanest proof. Pass it into the image as a build argument from the pipeline — do not hardcode it.

A.2 Linting ESLint

- ESLint configured at the project root (flat config `eslint.config.js` is preferred on ESLint 9+).
- Extend a real ruleset — the recommended config plus at least three rules you chose deliberately.
- Add a `lint` script to `package.json`. It must exit non-zero when there is a violation.
- Zero warnings and zero errors on the committed code. Run it with `--max-warnings=0`.

```
"scripts": {
  "lint": "eslint . --max-warnings=0",
  "test": "jest --coverage",
  "start": "node src/server.js"
}
```

A.3 Unit tests Jest or Vitest

- Minimum **8 meaningful unit tests**. Tests that only assert `true === true` score zero.
- Cover the happy path *and* the failure path of your business logic (bad input, missing item, etc.).
- At least two tests must hit the HTTP layer (`supertest` is the usual choice).

- Enforce a coverage threshold of **70% statements** in the test config so the pipeline fails when coverage drops.
- Tests must not require a network connection or a running server on a fixed port.
- Produce a JUnit-format report so GitLab can display test results on the MR page.

A.4 Dockerfile

- **Multi-stage build.** Install dev dependencies in a build stage; ship only production dependencies.
- Base the final stage on a slim or alpine image.
- Run as a **non-root user**.
- Accept a build argument for the commit SHA and expose it to the app as an environment variable.
- Include a **HEALTHCHECK** instruction hitting **/health**.
- Add a **.dockerignore** (at minimum **node_modules**, **.git**, coverage output).

4. Part B — AWS Infrastructure

You need **three EC2 instances** in the same region. Ubuntu 22.04 or 24.04 LTS is recommended.

Instance	Role	Must have
gitlab-runner	CI executor	Docker Engine, GitLab Runner package. Does <i>not</i> serve traffic. Needs outbound internet only.
app-dev	Development target	Docker Engine, a deploy user, inbound SSH from the runner and inbound HTTP.
app-prod	Production target	Same as dev, but treated as the protected environment.

B.1 Networking and security groups

- Security groups must be restrictive. **Do not open port 22 to 0.0.0.0/0**. SSH on the app servers should be reachable from the runner's security group (or its private/elastic IP) and, if you need it, from your own IP.
- Open the application port (or 80) as required for grading. Document what you opened and why.
- Give the app servers an Elastic IP, or otherwise ensure the address is stable — a public IP that changes on reboot will silently break your pipeline.

B.2 The deploy user and SSH keys

The pipeline must authenticate as a dedicated machine user, not as `ubuntu` with your personal key.

1. On each app server create a user (e.g. `deployer`) and add it to the `docker` group.
2. On your workstation generate **two separate key pairs** — one for dev, one for prod. Never reuse one key across environments.

```
ssh-keygen -t ed25519 -f ~/.ssh/deploy_dev -C "gitlab-ci-dev" -N ""
ssh-keygen -t ed25519 -f ~/.ssh/deploy_prod -C "gitlab-ci-prod" -N ""
```

3. Install the matching *public* key into `~deployer/.ssh/authorized_keys` on the matching server.
4. Capture each server's host key fingerprint for the pipeline:

```
ssh-keyscan -H <server-ip>
```

5. Disable password authentication in `sshd_config` on both app servers.

Forbidden. Any private key, password, token or IP committed to the repository is an automatic fail for the security portion of the grade, even if you delete it in a later commit (it stays in the Git history). If it happens, rotate the key immediately and say so in your report.

B.3 Install and register the GitLab Runner

1. Install Docker Engine and the `gitlab-runner` package on the runner instance.
2. In your GitLab project go to **Settings → CI/CD → Runners** and create a new project runner. Give it a tag such as `aws-runner` and note the authentication token.
3. Register it with the **Docker executor** and a sensible default image.
4. Confirm it shows as online in the Runners page, then disable shared/instance runners for the project so you can prove your own runner is doing the work.
5. You will need Docker-in-Docker (or a mounted socket) to build images. Decide which approach you are using and justify it in your report — there is a real security trade-off between them.

5. Part C — GitLab Environments and CI/CD Variables

C.1 Create the environments

Create two environments in **Operate → Environments**, named exactly `dev` and `prod`. Your deploy jobs must declare them with the `environment:` keyword and set a `url:` so the links appear in the GitLab UI.

C.2 Variable configuration graded closely

Every secret goes in **Settings → CI/CD → Variables**. Nothing sensitive may exist in `.gitlab-ci.yml`. Each variable must be scoped to the correct environment so the dev job physically cannot read production credentials.

Variable	Scope	Flags	Purpose
<code>SSH_PRIVATE_KEY</code>	<code>dev</code>	File type, Masked	Key for the dev deployer user
<code>SSH_PRIVATE_KEY</code>	<code>prod</code>	File type, Masked, Protected	Key for the prod deployer user
<code>SSH_KNOWN_HOSTS</code>	<code>dev</code> / <code>prod</code>	File type	Host key of the target server
<code>DEPLOY_HOST</code>	<code>dev</code> / <code>prod</code>	—	Target IP or DNS name
<code>DEPLOY_USER</code>	<code>dev</code> / <code>prod</code>	—	Machine user name
<code>APP_PORT</code>	<code>dev</code> / <code>prod</code>	—	Published host port
<code>REGISTRY_READ_USER</code> / <code>_TOKEN</code>	All	Masked	Deploy token with <code>read_registry</code> , so the app servers can pull images

Add anything else your app needs (a fake API key, a log level, etc.) — at least one non-secret variable that *differs* between dev and prod, to demonstrate scoping.

Two traps you will almost certainly hit.

1. A variable marked *Protected* is only exposed on protected branches and tags. Merge-request pipelines run on unprotected feature branches, so a protected dev key will appear empty and your job will fail with a confusing SSH error. Protect the prod key; do not protect the dev key.
2. An environment-scoped variable is only injected into jobs that declare that exact `environment:`. If your job does not declare it, the variable is simply not there.

C.3 Protect the main branch

Under **Settings** → **Repository** → **Protected branches**, mark `main` as protected and disallow direct pushes. All changes must arrive through a merge request. This is what makes the protected prod variable meaningful.

6. Part D — The Pipeline

D.1 Required stages

Stage	Job	Behaviour
lint	ESLint	Fails the pipeline on any error or warning.
test	Unit tests	Runs the suite, enforces coverage threshold, uploads a JUnit report and the coverage artifact.
build	Docker build & push	Builds the image, tags it with the commit SHA, pushes to the GitLab Container Registry. On <code>main</code> , additionally tag <code>latest</code> .
deploy	deploy_dev	Runs only on merge request pipelines . Deploys to <code>app-dev</code> .
deploy	deploy_prod	Runs only on <code>main</code> . Deploys to <code>app-prod</code> . Must be a manual job (<code>when: manual</code>) to model a release gate.

- Lint and test must run on every pipeline, including merge requests.
- Deploy jobs must not run if lint, test or build failed.
- All jobs must run on *your* runner (use its tag).

D.2 Image tagging

Use `$CI_REGISTRY_IMAGE` and tag with `$CI_COMMIT_SHORT_SHA`. Deploying a `latest` tag to production is not acceptable — the prod deploy must pull the exact immutable tag that was built for that commit, so a rollback is just re-running an older job.

```
docker login -u "$CI_REGISTRY_USER" -p "$CI_REGISTRY_PASSWORD" "$CI_REGISTRY"
docker build --build-arg GIT_SHA="$CI_COMMIT_SHORT_SHA" \
    -t "$CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA" .
docker push "$CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA"
```

D.3 The deploy step

Each deploy job must, over SSH, on the target server:

1. Log in to the GitLab registry using the read-only deploy token.
2. Pull the image tagged with this commit's SHA.
3. Stop and remove the previous container.
4. Start the new container with a restart policy, the correct published port, and the environment variables for that environment.
5. Poll `/health` until it returns `200`, with a timeout. **If the health check never passes, the job must exit non-zero** so the pipeline goes red.
6. Remove dangling images so the disk does not fill up.

Starting point for SSH setup inside a job

```
before_script:
  - apk add --no-cache openssh-client curl
  - eval $(ssh-agent -s)
  - chmod 600 "$SSH_PRIVATE_KEY"
  - ssh-add "$SSH_PRIVATE_KEY"
  - mkdir -p ~/.ssh && chmod 700 ~/.ssh
  - cat "$SSH_KNOWN_HOSTS" > ~/.ssh/known_hosts
```

Note: `SSH_PRIVATE_KEY` is a *File*-type variable, so the variable holds a path, not the key text. Using `StrictHostKeyChecking=no` instead of a real `known_hosts` file will cost you marks.

D.4 Skeleton `.gitlab-ci.yml`

This is a starting scaffold, not a solution. The `TODO` parts are yours to complete.

```
stages: [lint, test, build, deploy]

variables:
  IMAGE: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA

default:
  tags: [aws-runner]

lint:
  stage: lint
  image: node:20-alpine
  script:
    - npm ci
    - npm run lint

unit_test:
  stage: test
  image: node:20-alpine
  script:
    - npm ci
    - npm test
  artifacts:
    reports:
      junit: TODO
    paths: [coverage/]

build_image:
  stage: build
  image: docker:27
  services: [docker:27-dind]
  script:
    - TODO # login, build with --build-arg, push SHA tag
    - TODO # on main only, also tag and push :latest

deploy_dev:
  stage: deploy
  environment:
    name: dev
    url: http://$DEPLOY_HOST:$APP_PORT
```

```
rules:
  - if: $CI_PIPELINE_SOURCE == "merge_request_event"
script:
  - TODO    # ssh to $DEPLOY_USER@$DEPLOY_HOST and run the release

deploy_prod:
  stage: deploy
  environment:
    name: prod
    url: http://$DEPLOY_HOST:$APP_PORT
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
      when: manual
  script:
    - TODO
```

Workflow rules. By default you may see duplicate pipelines (one for the branch, one for the MR). Add a top-level **workflow:** block so that merge-request pipelines and **main** pipelines run, but detached branch pipelines do not. Getting this right is part of the grade.

7. Acceptance Test

Before you submit, run this exact sequence. If any step does not behave as described, you are not finished.

1. Create a branch, change the message returned by your resource endpoint, and open a merge request.
2. Expect: lint, test and build run. `deploy_dev` runs automatically. `deploy_prod` does not appear.
3. Open the dev URL. Expect: the new message, and `/version` showing the MR commit SHA.
4. Push a commit that breaks a lint rule. Expect: the pipeline fails at the lint stage and nothing is deployed.
5. Fix it, then merge the MR.
6. Expect: a pipeline on `main` with `deploy_prod` waiting for a manual click. Prod is still on the old version until you click it.
7. Click it. Expect: prod now serves the new message, and `/version` reports `prod` as the environment while dev still reports `dev`.
8. Check the Container Registry. Expect: one image per build, tagged by SHA.

8. Grading

Criterion	Points	Notes
Application: endpoints, structure, correctness	10	
ESLint configured and enforced in CI	10	
Unit tests: quality, coverage threshold, JUnit report	15	Trivial tests score 0
Dockerfile: multi-stage, non-root, healthcheck, build arg	10	
AWS: three instances, sane security groups, deploy user	10	
Self-hosted runner registered and actually used	10	
Pipeline logic: stages, MR→dev, main→prod, manual gate	15	
Secrets handling: scoping, masking, protection, no leaks	10	
Registry usage and immutable SHA tagging	5	
README and report quality	5	
Total	100	

Bonus (up to +10)

- A working rollback job that redeploys a previous SHA on demand.
- Zero-downtime deployment (start the new container, health check it, then switch traffic).
- A secret-scanning or dependency-audit job in the pipeline.

- Infrastructure defined as code (Terraform or CloudFormation) instead of clicking in the console.
- A reverse proxy in front of the app with a real hostname.

Automatic penalties

- Any credential committed to the repository: −20 and a mandatory rotation note in the report.
- SSH open to the world on the app servers: −5.
- Deploy jobs that ignore a failing health check: −5.
- Using shared runners instead of your own: −10.

9. Troubleshooting Hints

- **"Permission denied (publickey)"** — usually the protected-variable trap, a key with wrong permissions, or the public key installed for the wrong user's `authorized_keys`.
- **Variable is empty in the job** — check the environment scope matches the job's `environment: name:` exactly, and check the Protected flag against the branch.
- **Masking refuses to enable** — GitLab only masks values meeting certain length and character requirements. Base64-encode the value, or use a File-type variable.
- **Image pull fails on the app server** — the job token is not valid outside the job. Use a deploy token with `read_registry`.
- **Docker build fails in the job** — missing dind service, or `DOCKER_HOST` / TLS settings not configured for your runner's executor.
- **Disk full on the runner after a few days** — prune old images and build cache.

Ask questions early, especially about AWS networking and variable scoping — those two consume the most time. Bring your pipeline logs when you ask. Remember to shut down your instances when you are done.